VisionAssist: Comprehensive Documentation

Group 2 – Data Science & AI Lab (BSCSS3001)

A. Overview

1. Purpose

VisionAssist is an assistive system designed to help visually impaired users by providing **real-time, voice-based navigation cues** derived from computer vision. The system captures video frames, detects obstacles using YOLOv8, generates short natural-language messages describing the scene, converts the message into speech using online TTS (gTTS), and plays the audio to the user.

The system aims to:

- Detect obstacles in real time
- Identify object type and approximate spatial location
- Provide concise audio warnings
- Operate with minimal latency

2. System Architecture

```
Camera / Video Input
        |
        ▼
Frame Capture (OpenCV)
        |
        ▼
Object Detection (YOLOv8)
```

```
        |
        ▼
Post-processing & Spatial Reasoning
        |
        ▼
Message Generator (Python)
        |
        ▼
Online TTS (gTTS) → MP3 File
        |
        ▼
Playsound (Audio Output)
```

3. Deployed Components

- **Model**: YOLOv8n (Ultralytics)
- **Inference Engine**: Python + OpenCV + PyTorch
- **Text-to-Speech**: gTTS (Google Text-to-Speech)
- **Audio Output**: playsound library
- **Execution**: Local CPU / laptop runtime
- **Deployment Mode**: Fully local vision pipeline + cloud-based TTS

B. Technical Documentation

1. Environment Setup

Languages & Frameworks

- Python 3.10+
- PyTorch 2.x
- Ultralytics YOLOv8
- OpenCV
- NumPy
- gTTS (online TTS)
- playsound
- Pandas, pyarrow (for earlier dataset handling)

Hardware

- Training environment: GPU (Tesla T4)
- Deployment environment: CPU (Intel i7 or similar)

Installation

```
pip install -r requirements.txt
```

Optional (if using Conda):

```
conda env create -f environment.yml
```

Key Dependencies

```
ultralytics
torch
opencv-python
gTTS
playsound
numpy
pandas
```

2. Data Pipeline

Datasets Used

- **COCO subset**: ~5,000 images with common street/indoor objects
- **Custom dataset**: 2,138 first-person navigation frames (YouTube sources)

Data Splits

- 70% training
- 20% validation
- 10% testing

Preprocessing

- Resize images → 640×640
- YOLO-format annotations
- Augmentations (Mosaic, flips, HSV, scaling)

Classes

- person
- obstacle
- vehicle
- door
- stair
- furniture

Challenges

- Domain mismatch
- Small-object detection
- Motion blur

3. Model Architecture

Core Model: YOLOv8n

- Lightweight
- Good for real-time inference on edge devices
- Supports 640×640 input

Outputs

- Bounding boxes

- Class IDs
- Confidence scores

Model Features

- Multi-scale feature extraction
- Faster inference than YOLOv5n
- CIoU loss for bounding box accuracy

4. Training Summary

Training Environment

- Ultralytics framework
- PyTorch backend
- Mixed precision enabled
- Early stopping applied

Best Hyperparameters (Optuna)

Parameter	Value
Learning Rate	0.0003925
Momentum	0.937
Weight Decay	0.00211
Epochs	15
Batch Size	64

Performance Metrics

Metric	Value
mAP50	0.889
mAP50-95	0.805
Precision	0.834
Recall	0.797

5. Evaluation Summary

Quantitative

Metric	Value
mAP50	0.859
mAP50-95	0.797
Precision	0.834
Recall	0.797
End-to-end latency	~0.15s

Observations

- Objects up to 5–6 meters detected
- YOLOv8n performs adequately on CPU
- Fewer false positives after temporal filtering

6. Inference Pipeline (Aligned With Deployment Code)

Real-time Flow

```
1. Capture frame from webcam
2. Resize, normalize
3. YOLOv8 model inference
4. For each detection:
      - Compute spatial position (left/middle/right)
      - Estimate distance (approx from bbox size)
5. Create simple English sentence
6. TTS: gTTS synthesizes speech (requires Internet)
7. Save as temporary MP3 (tts_output.mp3)
8. playsound plays audio to user
```

Example Code Path

```
detections → analyze_positions() → generate_message() →
gTTS(text).save → playsound
```

Performance Notes

- gTTS introduces **network latency** (200–600ms)
- Detection part runs locally (<80 ms on CPU)

7. Deployment Details

Running the System Locally

- `python visionassist_inference.py --source <input_source>`

Inputs Supported

- Webcam stream (`--source webcam`)
- Video files (`--source path/to/video.mp4`)
- Image folder (`--source path/to/images/`)

Outputs

- Bounding box visualizations (on video/image frames)
- Voice alerts via gTTS
- Console logs for debugging

TTS Requirements

- Internet connection is mandatory
- Text is sent to Google's TTS API
- MP3 is generated per message

Notes

- For real-time streaming with the camera, it's recommended to use the WebSocket endpoint (`/ws/camera/stream`) for smooth annotated frame + audio streaming.
- The local script is mainly for debugging, testing, or non-server use.

8. System Design Considerations

Why Online TTS?

- Natural voice
- No heavy local models
- Works with minimal CPU load

Trade-offs

Pros	Cons
High-quality audio	Requires internet
Lightweight	Additional latency
Easy to implement	Privacy considerations

Latency Reduction Strategies

- Avoid redundant messages
- Only trigger TTS on meaningful changes
- Message throttling (cooldowns)

9. Error Handling & Monitoring

Failure Modes

Failure	Handling
TTS network failure	Skip audio, log warning
playsound error	Print fallback message

No detections	Silent frame
Camera failure	Hard exit with error

Logging

- Frame-by-frame console logs
- Detection metadata

10. Reproducibility Checklist

To reproduce the system:

Required

- Python 3.10
- gTTS (online TTS)
- Ultralytics model weights `best.pt`
- Camera access

Consistent Results

- Use same YOLO checkpoint
- Use same frame size (640×640)
- Stable internet for consistent audio generation

C. User Documentation (For Non-Technical Users)

App Overview

The VisionAssist System is a **real-time object detection** and **audio-aler**t application designed to help users understand their surroundings. It uses live camera feed from the browser, identifies nearby objects, estimates their distance, and provides spoken alerts or narration.

Use Cases:

- Assisting visually impaired users in outdoor environments
- Informing users about nearby obstacles
- Real-time awareness in low-visibility areas

Input Description

Users only need to allow camera access in their browser. The application automatically streams video frames to the server over WebSocket.

No manual input is required.

Output Description

Users will receive:

1. Annotated Live Video

- Bounding boxes around objects
- Distance and direction (e.g., “Car 8m Right”)

2. Audio Alerts

- Alerts when an object is close
- Example: *“Caution: Car, 7 meters, Right.”*

3. Continuous Narration (Optional)

If no threat is detected, the system gives short narration every 2 seconds.

Step-by-Step Instructions

How to Launch the App

1. Visit the provided HTTPS website (example): https://YOURDOMAIN/
2. The page will ask for camera permission → click Allow.
3. Live video streaming starts automatically.

How to Interact with It

- Just point your phone or laptop camera in front of you.
- You will see a live annotated feed.
- You will hear spoken alerts whenever a nearby object is detected.

Example Interactions

Scenario	System Behavior
A car approaches from the right	“Caution: Car, 6 meters, Right.”
A static object on left	“Bench left, 12 meters.”
No objects nearby	“Area clear.”

Troubleshooting

No video visible

- Ensure your browser has camera permissions enabled.
- Refresh the page.
- Try switching browsers (Chrome preferred).

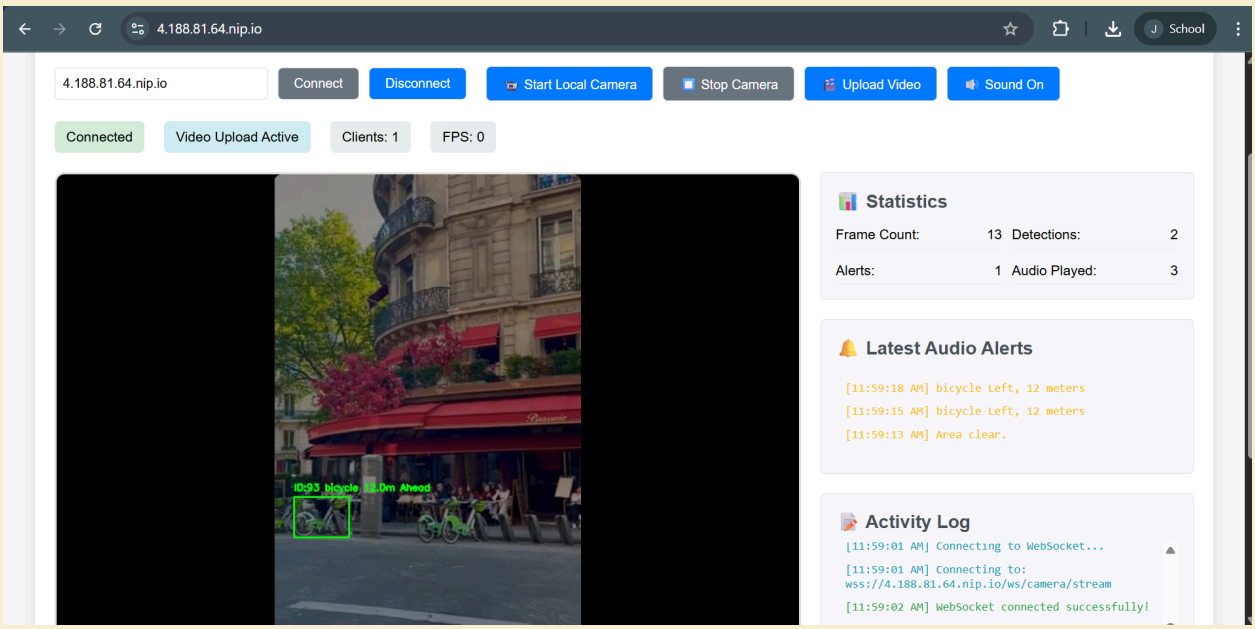
No audio alerts

- Check your device volume.
- Ensure browser autoplay permission is allowed.

“WebSocket disconnected” error

- Weak internet connection
- Server temporarily unreachable
- Try reloading the page

Screenshots



D. API Documentation (For Technical Users)

Base URL

```
None
http://<server-ip>:8000
```

Example from deployment:

```
None
http://4.188.81.64:8000
```

1. Health Check

GET /health

Description: Checks if the server and YOLO model are loaded.

Response Example:

```
JSON
{
  "status": "healthy",
  "pipeline_initialized": true,
  "model_loaded": true
}
```

2. WebSocket Endpoint

ws://:8000/ws/camera/stream

Used for real-time video frame uploads from browser and returning:

- Annotated frame
- Base64 encoded audio alerts
- Alert texts

Incoming Message Format

```JSON
{
  "command": "process_frame",
  "frame_data": "<base64_jpeg>"
}
```

Outgoing Response

```JSON
{
  "type": "processing_result",
  "frame_data": "<base64_annotated_jpeg>",
  "audio_data": ["<base64_mp3>"],
  "alert_texts": ["Caution: Car, 7 meters, Right."],
  "frame_count": 102,
  "detections_count": 3,
  "alerts_count": 1,
  "timestamp": 1732000000
}
```

3. Performance Logs

GET /performance/stats

Returns average FPS, timing stats, detection count etc.

POST /performance/flush

Flushes logs to CSV.

GET /performance/download

Downloads the CSV file.

4. HTTP Root Page

GET /

Returns the WebSocket HTML client.

Example cURL Request

```shell
curl -X GET http://<server-ip>:8000/health
```

E. Licensing & Dataset References

1. Project License

This project is released under the **MIT License** for research and educational use.

2. Dataset Source and License

- **Dataset:** COCO 2017 (Common Objects in Context)
- **Provider:** Microsoft COCO Consortium
- **Source:** COCO Dataset
- **License:** CC BY 4.0 (link)

Usage Permissions: Share, adapt, and use commercially with proper attribution. No additional restrictions may be applied.

Attribution: Proper credit to Microsoft COCO Consortium is included in documentation and outputs.

3. Data Integrity & Ethical Use

- **No PII:** COCO contains no identifiable personal data; custom datasets are screened for faces/identifiers.
- **Ethical Standards:** Use limited to research/educational purposes; ensures fairness, accountability, transparency.
- **Bias Awareness:** Potential dataset imbalances are documented and mitigated during evaluation.

4. Data Usage & Compliance

Aspect	Description
License	CC BY 4.0
Permission	Use, share, modify with attribution
Redistribution	Permitted with credit
Commercial Use	Allowed
Adaptation	Allowed with credit
PII	None
Storage & Access	Secure, controlled environment
Retention	Project duration only; deleted post-submission

Compliance Procedure: Verified license, validated intended use, maintained attribution, and ensured ethical/legal standards throughout the project lifecycle.

5. Summary & Declaration

The dataset is used responsibly under CC BY 4.0 with proper attribution. No PII is included, and all ethical, legal, and compliance measures have been followed.

F. Future Work / Maintenance Notes

1. Possible Extensions

- **New Datasets:** Incorporate indoor navigation datasets or other visually impaired navigation corpora to improve domain coverage.
- **Enhanced Models:** Experiment with YOLOv8 variants, Transformer-based detectors, or lightweight models for edge devices.
- **Additional Features:**
    - Haptic feedback integration to complement audio alerts
    - Multi-camera support or panoramic stitching for wider FOV
    - Velocity-aware alert prioritization for moving obstacles

2. Known Limitations

- **Performance:** Detection may degrade under low light, fast-moving objects, or crowded scenes.
- **Coverage:** Current model primarily trained on COCO; may misclassify rare or domain-specific obstacles.
- **Hardware Constraints:** Real-time processing depends on GPU availability; latency increases on CPU-only systems.

3. Model Retraining / Updates

- Update weights using new annotated datasets or augmented COCO samples.
- Hyperparameter tuning can be done via Optuna as in the current pipeline.
- Steps:

1. Prepare dataset in YOLO format
2. Fine-tune existing YOLOv8 model
3. Validate on representative frames and metrics (MAE distance, alert latency)
4. Replace `yolov8n_optuna_best.pt` and restart system service

4. Contacts / Maintainers

Project Team: VisionAssist (Group 2 – DSAI)

- **Karan Patil** – 22f2001061
- **Prashasti Sarraf** – 21f1001153
- **Tanuja Nair** – 21f1000660
- **Jivraj Singh Shekhawat** – 22f3002542
- **Balasurya K** – 22f3002744

Affiliation: DSAI Lab, BSCSS3001, IIT Madras BS Program

Support / Issues: Via Git repository issue tracker